



Deep Learning

Lectures on Generative models
UG-4, Aug 2025-26
Shaik Mohammad Rafi B

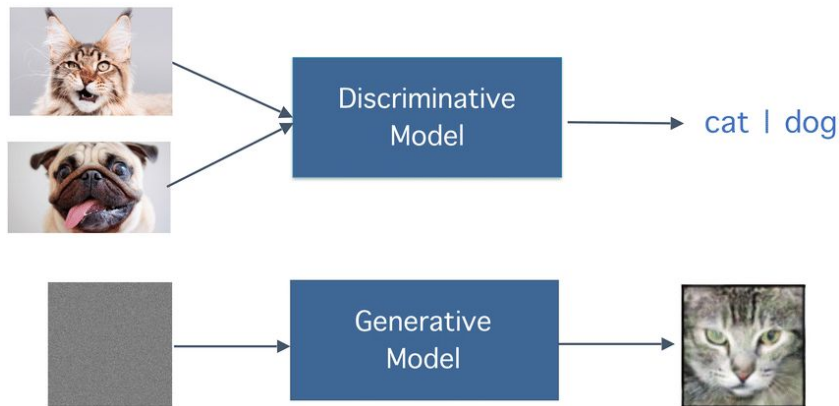
Machine Learning Models

1. Discriminative Models

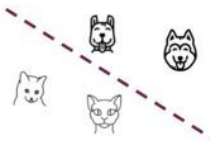
- Are supervised learning tasks.
- classifiers that attempt to identify the relationships between features and labels

2. Generative Models

- Reverse-engineer: label to datapoint
- Text generation
- Audio generation
- Image generation
- Data augmentation



Discriminative models



Features Class

$$X \rightarrow Y$$

$$P(Y|X)$$

Generative models



Noise Class Features

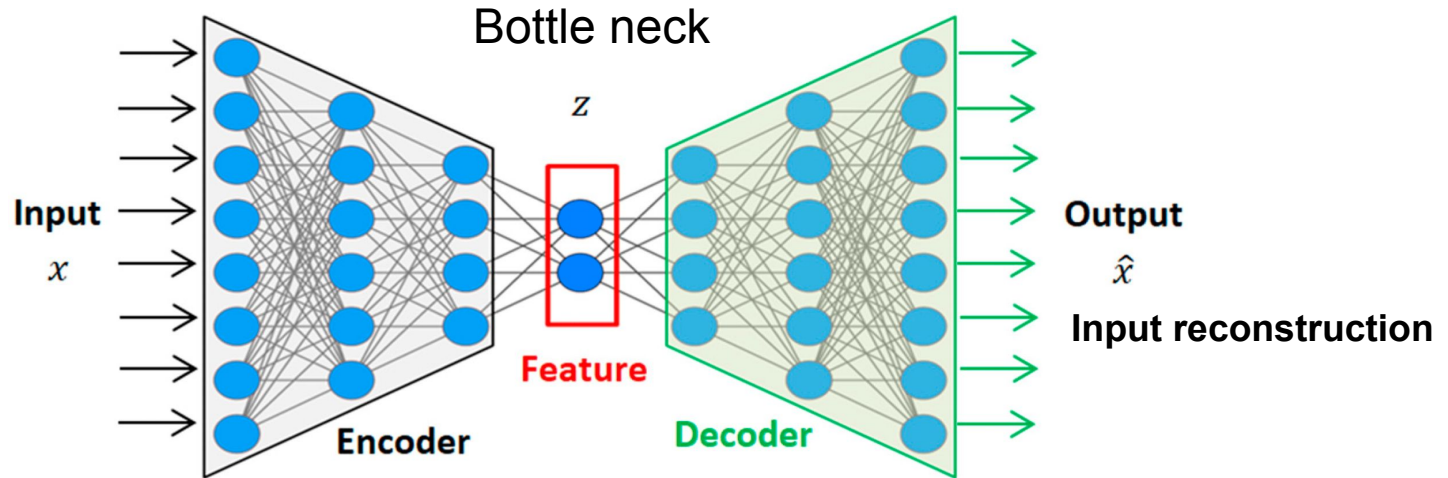
$$\xi, Y \rightarrow X$$

$$P(X|Y)$$

Auto Encoders

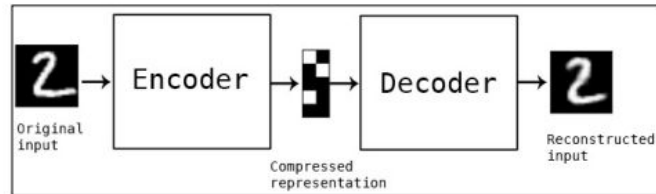
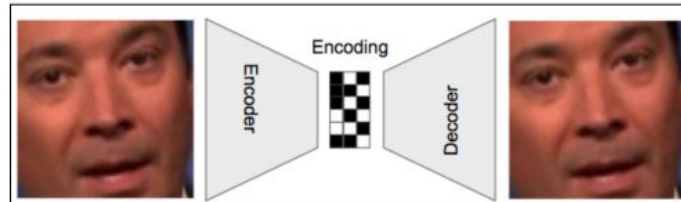
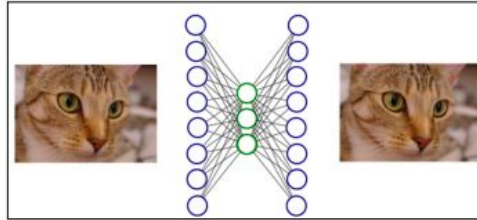
Data compression (dimensionality reduction):

- Linear model: PCA
- Non Linear model: Auto Encoder

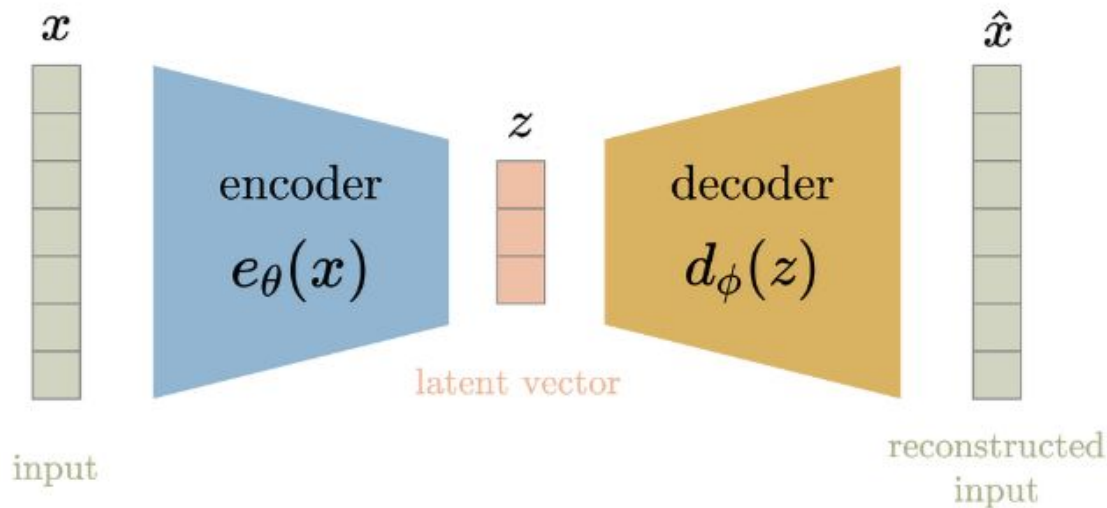


Auto Encoder

- A neural network trained using unsupervised learning
 - Trained to copy its input to its output
 - Learns an embedding



Training AE



$$\begin{aligned} loss &= \|x - \hat{x}\|_2 \\ &= \|x - d_{\phi}(z)\|_2 \\ &= \|x - d_{\phi}(e_{\theta}(x))\|_2 \end{aligned}$$

Minimize *loss* w.r.t encoder parameters θ and decoder parameters ϕ

Auto Encoder

- Extracts a compact low-dimensional vector: Embedding
- Trained by reconstruction the input (approximate).
- Mean squared error loss is computed between input and reconstruction
- Only able to compress data similar to what they have been trained on.

Autoencoders fail to learn anything useful:

- Capacity of encoder/decoder is too high (Capacity controlled by depth).
- Hidden code z has dimension equal to input x
- Overcomplete case: where hidden code z has dimension greater than input x

Data sampling

Suppose the dataset is:

$$X = \{2, 3, 5, 6, 7, \dots, 100\} \quad (\text{just 100 integers}) \quad \text{Uniform distribution}$$

- Empirical: pick a random element, say $x_{42} = 43$.

- Gaussian fit:

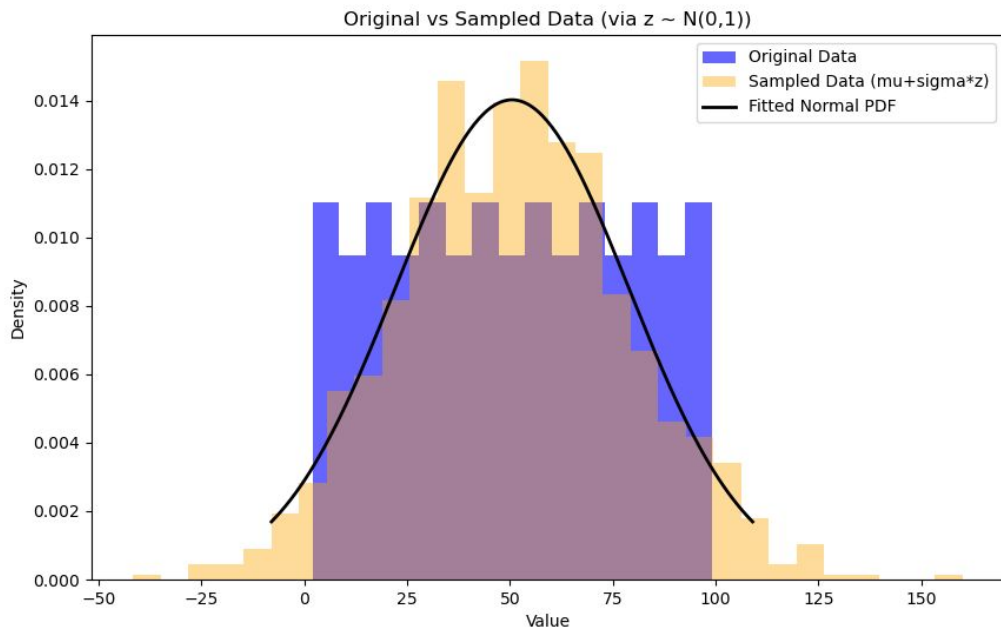
- Mean $\hat{\mu} = 51$

- Std $\hat{\sigma} \approx 29$

Sample: $x = \mu + \sigma z$,

$$x = 51 + 29 \cdot z, \quad z \sim \mathcal{N}(0, 1)$$

If $z = 0.5$, then $x = 51 + 14.5 = 65.5$.



Data generation

Linear Transformation (Normal Sampling)

- Start with $z \sim \mathcal{N}(0, 1)$ (a simple, well-known distribution).
- Apply a transformation $x = \mu + \sigma z$.
- Result: $x \sim \mathcal{N}(\mu, \sigma^2)$.
- Here, the generator is just a simple *linear function*:

$$G(z) = \mu + \sigma z.$$

$$z \sim \mathcal{N}(0, 1)$$



$$x = \mu + \sigma z$$



$$x \sim \mathcal{N}(\mu, \sigma)$$

Also called
reparameterization

Nonlinear Transformation (Complex Data)

- Real-world data (e.g., images, speech, text) has **highly complex, multimodal distributions**, not as simple as Gaussian.
- To generate such data, we still start with a simple latent variable $z \sim p(z)$ (commonly standard Gaussian or uniform).
- Instead of a fixed formula, we let a neural network G_θ learn the mapping:

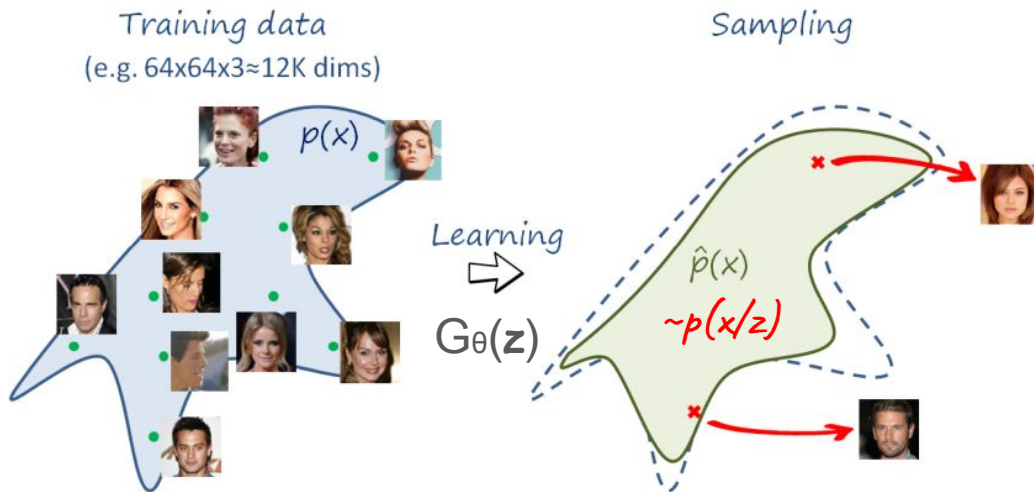
$$x = G_\theta(z).$$

- Now, G_θ can be deep and nonlinear, allowing it to warp the simple latent distribution into something as rich as natural images.

$$x = G_\theta(z) ; \theta = \{\mu, \sigma\}$$

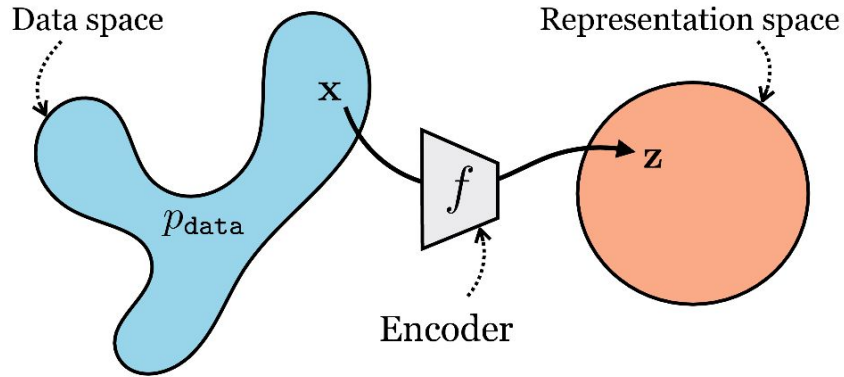
Generative Models

- Creates new data points.
- Learn the data distribution of an input training set, to generate new data points similar to the initial training set.

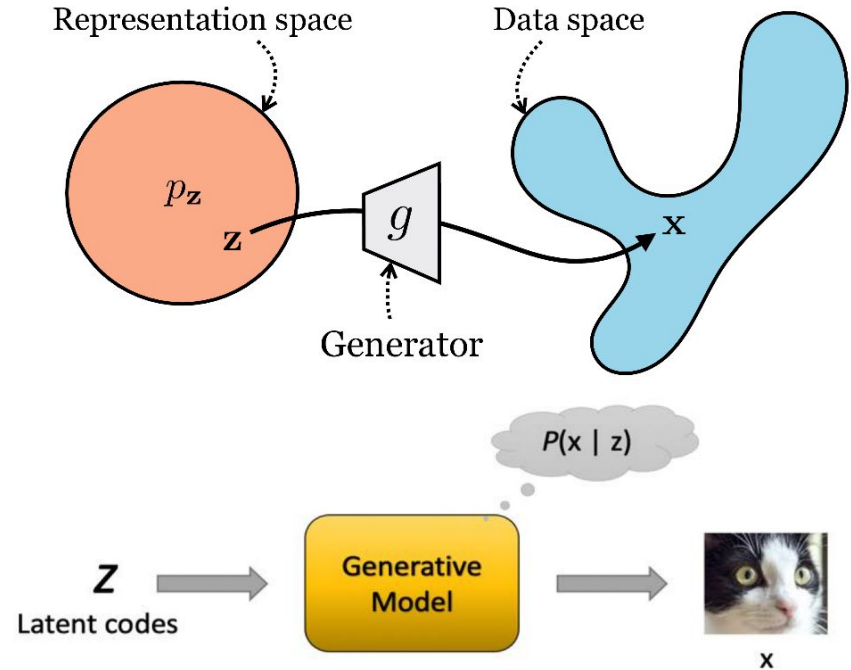


Auto encoders for data generation

Representation learning

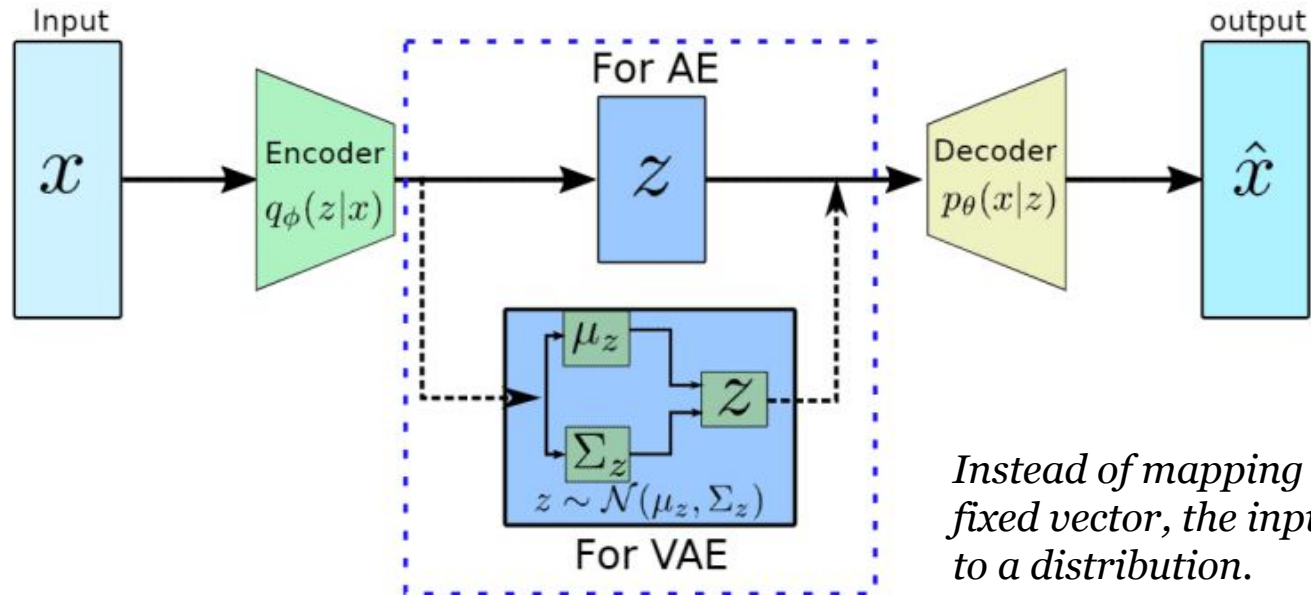


Generative modeling

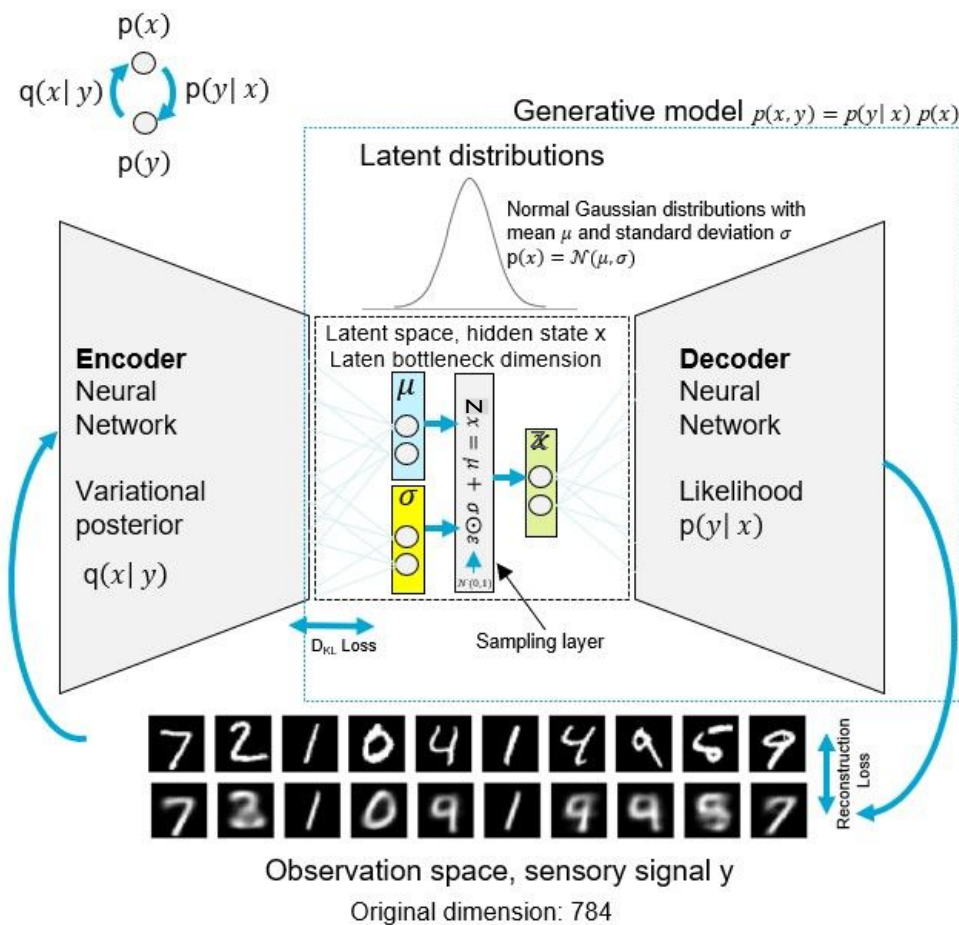


Variational AutoEncoder (VAE)

Learns a *probabilistic* latent distribution, enabling sampling and generation of new data, not just reconstruction.



Instead of mapping an input to a fixed vector, the input is mapped to a distribution.



Training VAE

VAE loss combines two terms:

1. **Reconstruction loss** (how well decoder reconstructs x).
2. **KL divergence regularizer** (forces latent codes z to stay close to the unit normal prior $\mathcal{N}(0, I)$).

$$\mathcal{L}_{VAE} = \underbrace{-\mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)]}_{\text{Reconstruction loss}} + \underbrace{D_{KL}(q_{\phi}(z|x) \parallel p(z))}_{\text{KL divergence regularizer}}$$

While Training

- The encoder outputs μ and $\log \sigma$
- We sample ϵ from $\mathcal{N}(0, I)$
- Compute $z = \mu + \sigma \epsilon$. (**reparameterization trick** to enable gradient propagation to encoder.)
- Pass z into the decoder to reconstruct x .
- Compute reconstruction + KL losses.
- Backpropagate normally.

Why reparameterization :

Direct sampling detaches network from μ and σ

$z = \text{normal}(\mu, \sigma)$. z is a vector (calls a random number generator) and $\frac{\partial L}{\partial \mu}$, $\frac{\partial L}{\partial \sigma}$ becomes zero.

reparameterization

$z = \mu_\phi(x) + \sigma_\phi(x) \cdot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ z is now differentiable function of μ, σ

ϵ which is independent of μ and σ

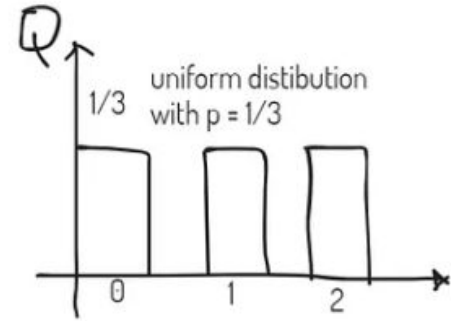
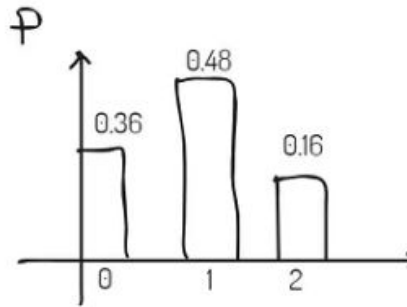
$$\frac{\partial L}{\partial \mu} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \mu}$$

$$\frac{\partial L}{\partial \sigma} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \sigma}$$

KL divergence

- Measure of how one probability distribution is different from the other.
- Not symmetric measure.

$$D_{KL}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

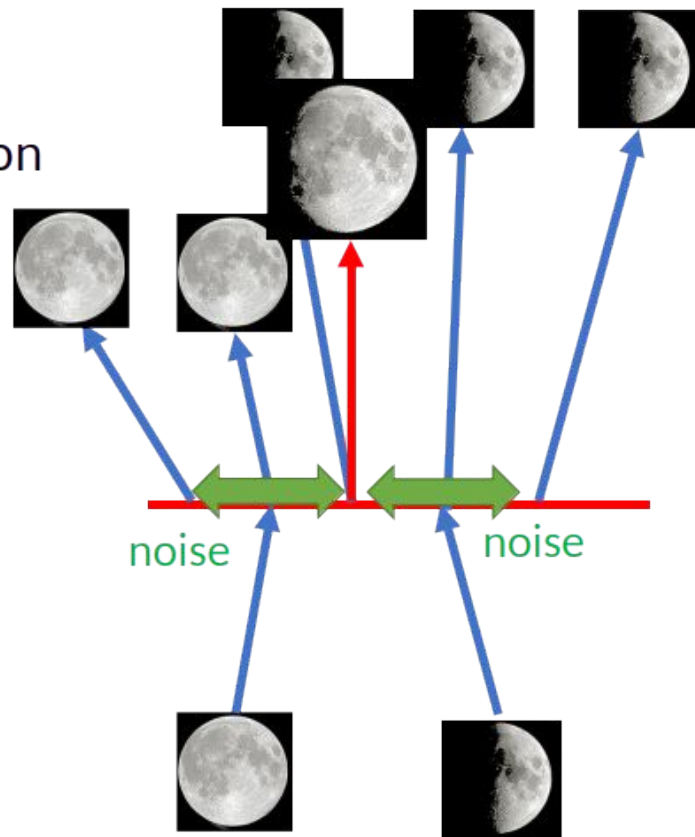
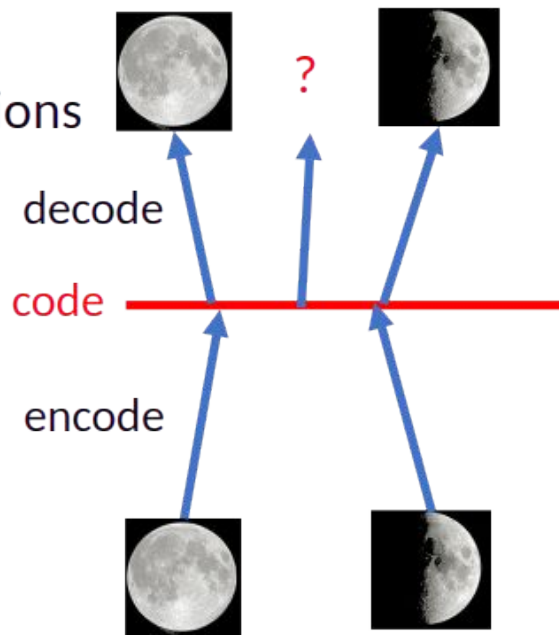


$$D_{KL}(Q || P) = \frac{1}{3} \ln \left(\frac{0.333}{0.36} \right) + \frac{1}{3} \ln \left(\frac{0.333}{0.48} \right) + \frac{1}{3} \ln \left(\frac{0.333}{0.16} \right)$$

$$= 0.14 \text{ nats}$$

What a VAE does

- Generate samples from a data distribution
- For any data
- Cool applications



VAE outputs



Samples from a VAE trained on MNIST

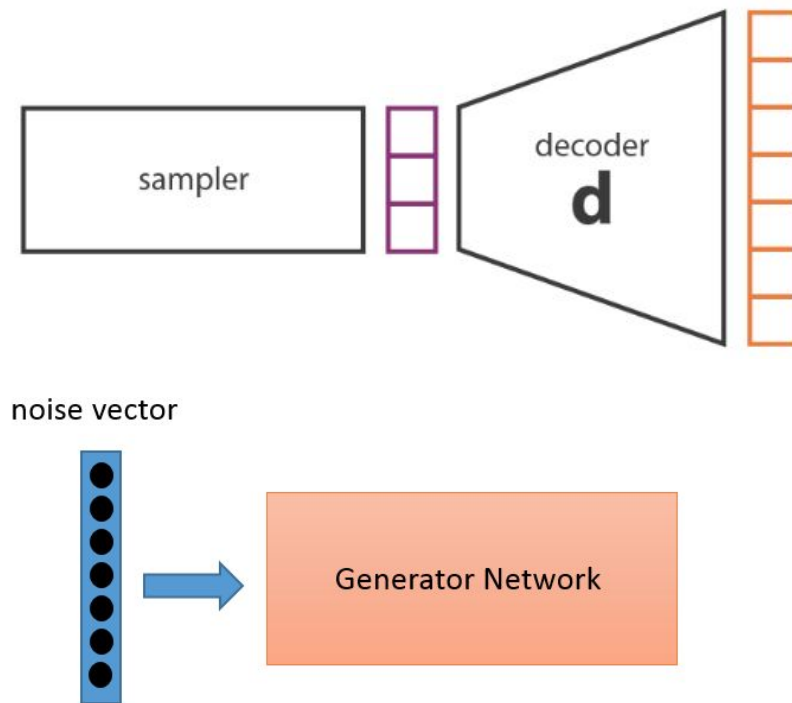


Samples from a VAE trained on a faces dataset

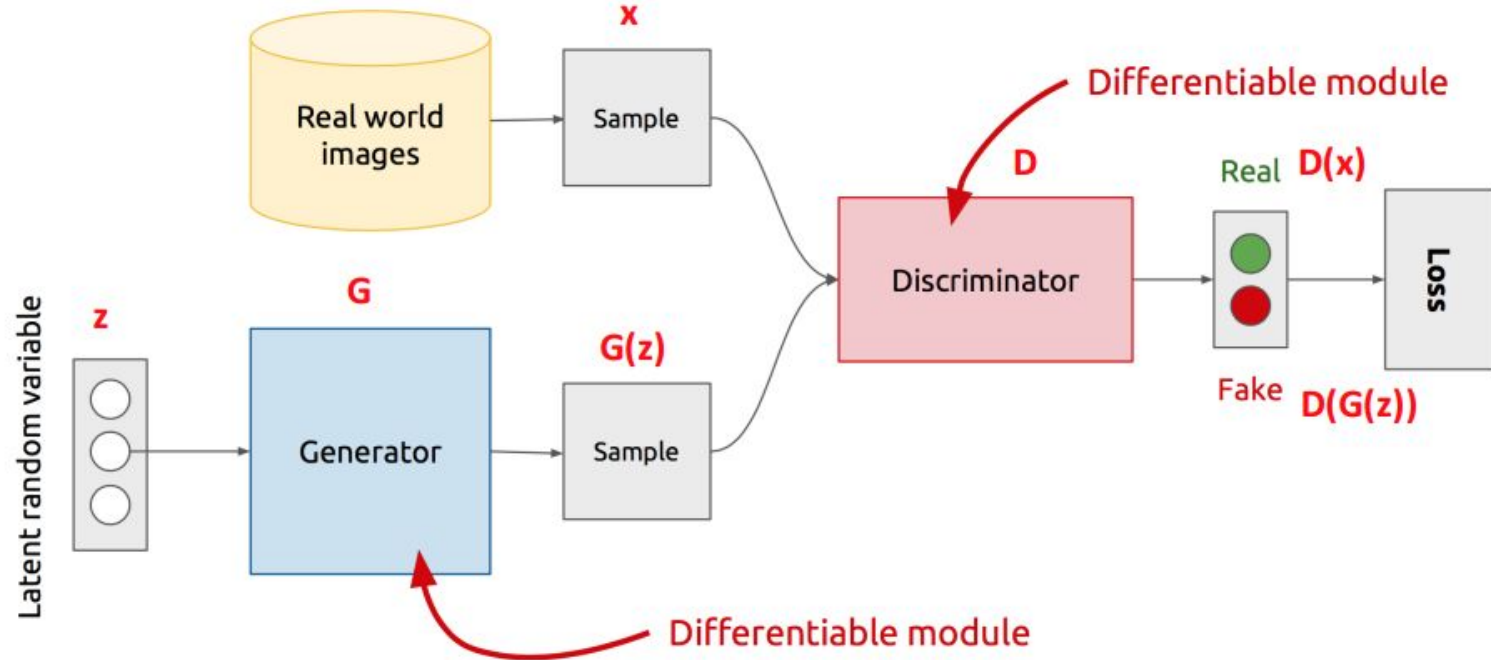
Generative Adversarial Network (GAN)

VAE limitations

- VAEs are theoretically complex.
- Don't generalize very well
- VAEs directly approximate $P(X)$ using latent variables z , which is complicated

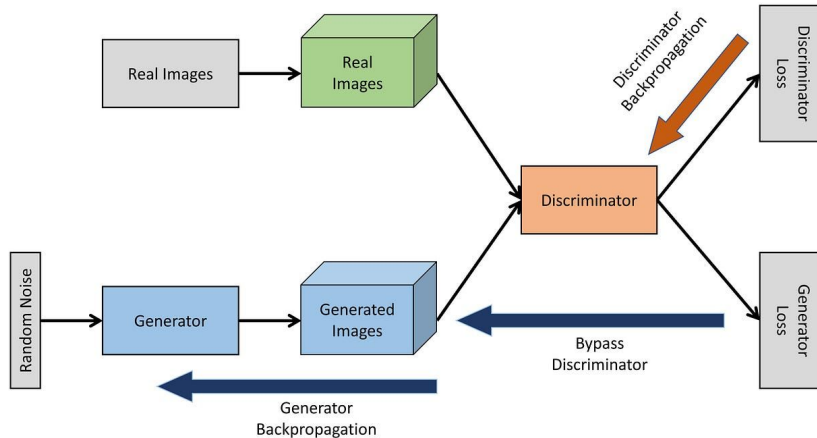


GAN Architecture

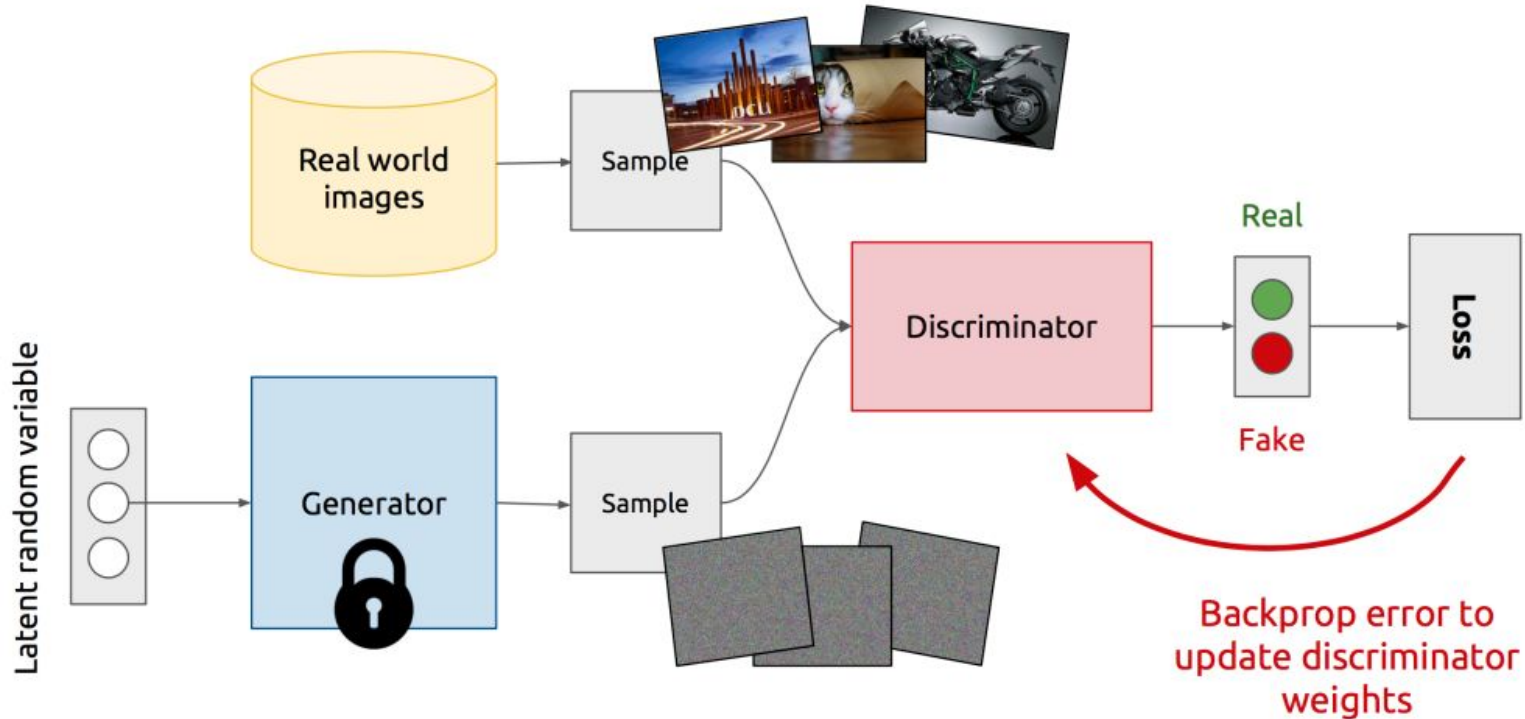


Training GANs

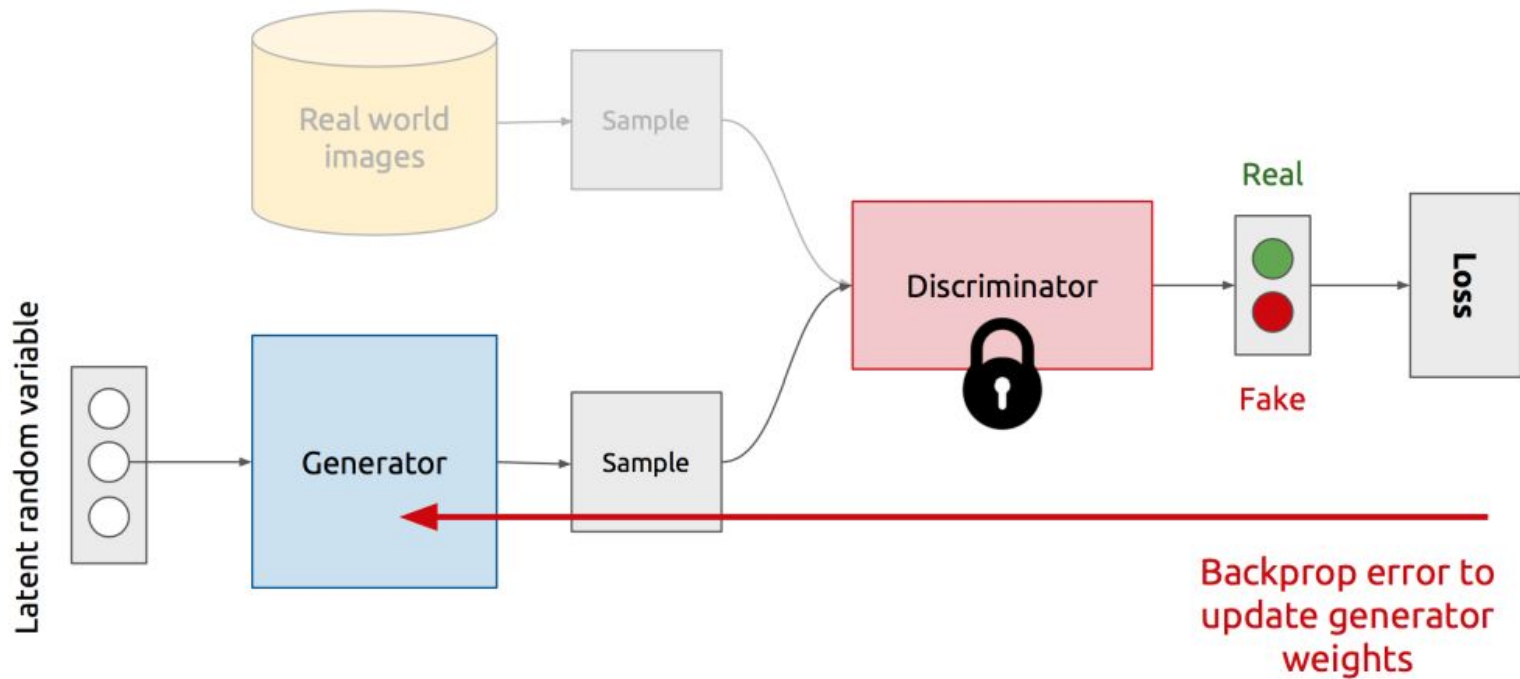
- **Generator:** generate fake samples, tries to fool the Discriminator
- **Discriminator:** tries to distinguish between real and fake samples
- **Adversarial training:** Train them against each other
- Repeat this and we get better Generator and Discriminator



Training Discriminator



Training Generator



Training loss GAN

$$\min_G \max_D V(D, G)$$
$$V(D, G) = \mathbb{E}_{x \sim p(x)} [\log D(x)] + \mathbb{E}_{z \sim q(z)} [\log(1 - D(G(z)))]$$

$$D^* = \underset{D}{\operatorname{argmax}} V(D, G)$$

$$G^* = \underset{G}{\operatorname{argmin}} V(D, G)$$

- D tries to **maximize** this loss — distinguish real vs fake correctly.
That is $D(x) = 1$ (real sample) and $D(G(z)) = 0$ (fake sample)
- G tries to **minimize** it — generate realistic fakes that fool DDD.
 $D(G(z)) = 1$ (fool discriminator with fake sample)

GAN's formulation

$$\min_G \max_D V(D, G)$$

- It is formulated as a **minimax game**, where:
 - The Discriminator is trying to maximize its reward $V(D, G)$
 - The Generator is trying to minimize Discriminator's reward (or maximize its loss)

$$V(D, G) = \mathbb{E}_{x \sim p(x)} [\log D(x)] + \mathbb{E}_{z \sim q(z)} [\log(1 - D(G(z)))]$$

- The Nash equilibrium of this particular game is achieved at:
 - $P_{data}(x) = P_{gen}(x) \quad \forall x$
 - $D(x) = \frac{1}{2} \quad \forall x$

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log \left(1 - D(G(z^{(i)})) \right) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D(G(z^{(i)})) \right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Discriminator
updates

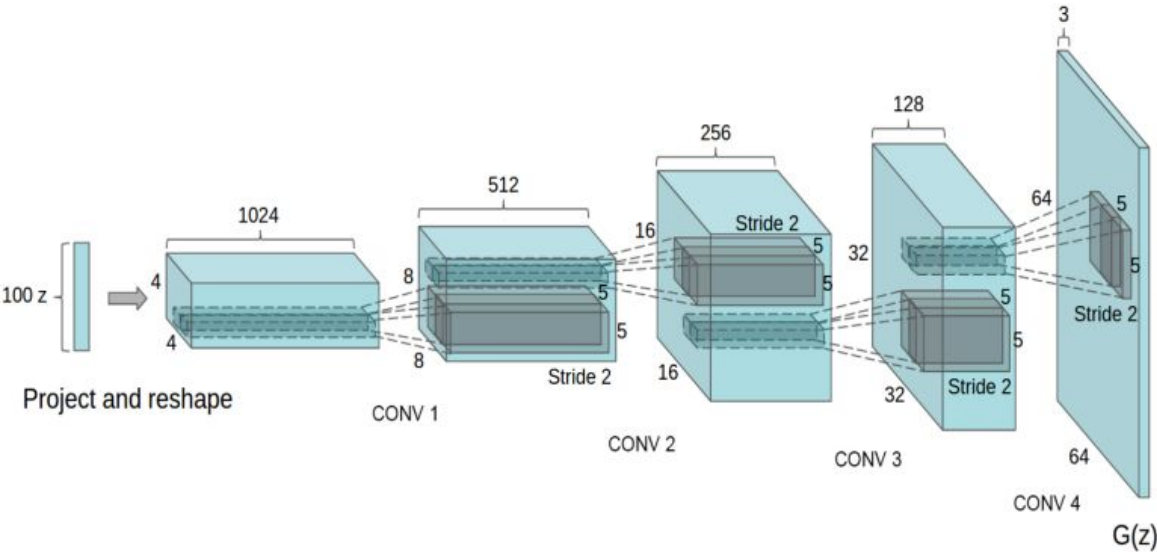
Generator
updates

Faces



Deep Convolutional GANs (DCGANs)

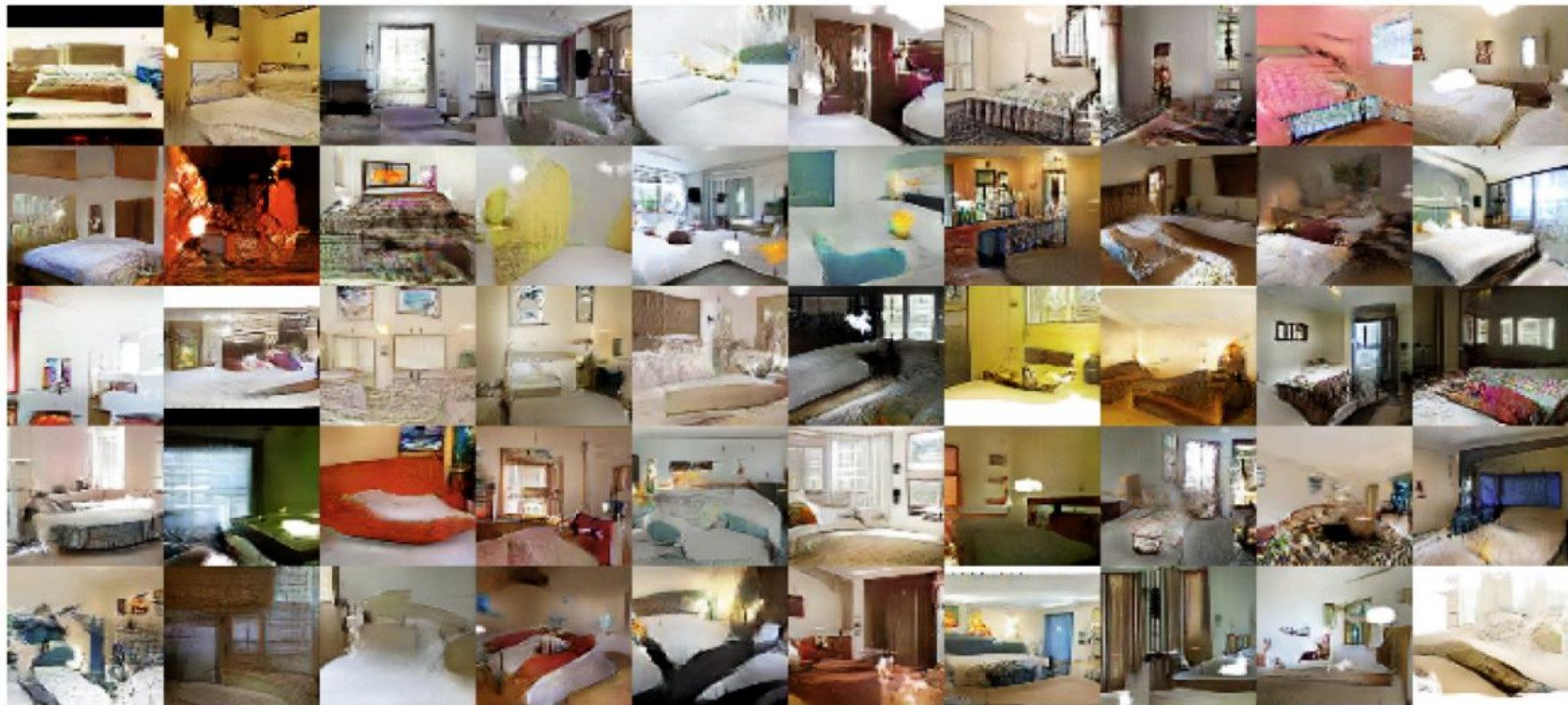
Generator Architecture



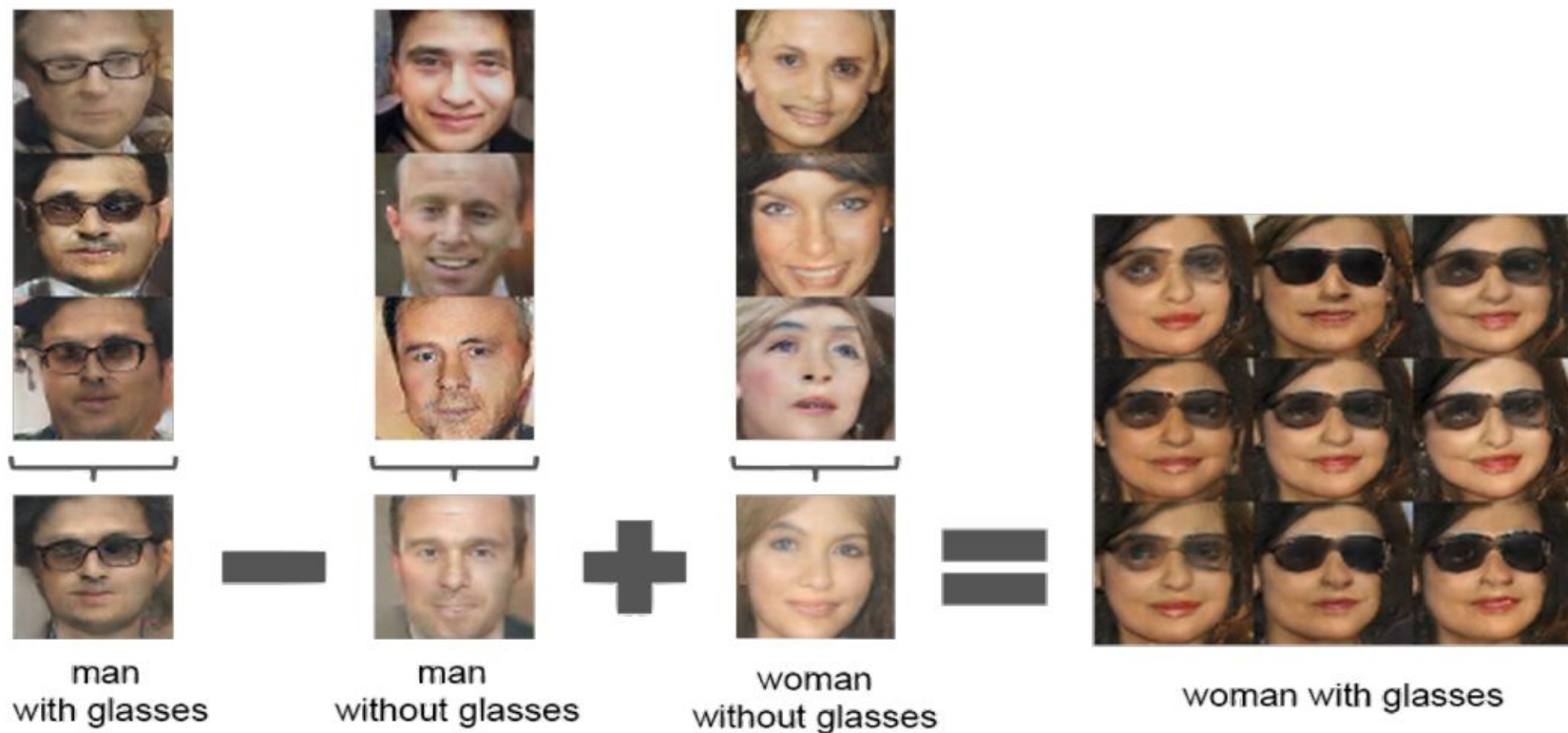
Key ideas:

- Replace FC hidden layers with Convolutions
 - **Generator:** Fractional-Strided convolutions
- Use Batch Normalization after each layer
- **Inside Generator**
 - Use ReLU for hidden layers
 - Use Tanh for the output layer

DCGAN: Bedroom images



Latent vectors capture interesting patterns...



Cycle GAN

Image-to-image translation without paired data.

For example:

- Translate horses $\leftrightarrow$ zebras
- Summer $\leftrightarrow$ winter landscapes
- Day $\leftrightarrow$ night scenes

all without having exact pairs of matching images.

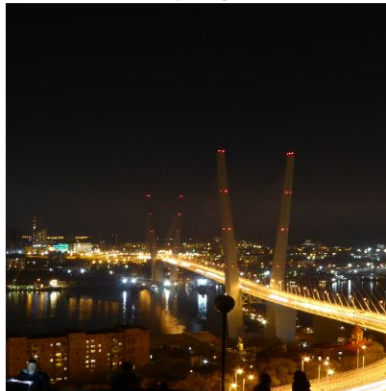
Input Image



Predicted Image



Input image



Generated image



Style GAN

[styleGAN demo](#)



Evaluating and improving generative models

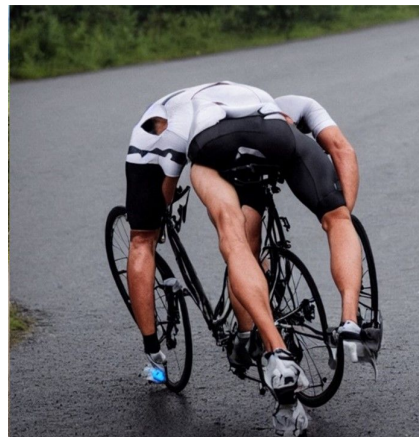
A good generative model should produce images that are

- Faithful/Realistic
- High Quality
- Diverse
- Semantically meaningful

Metrics to evaluate generative models

- FID (Fréchet Inception Distance) - Measures quality and diversity
- IS (Inception Score) - Measures quality and diversity
- WD (Wasserstein Distance) - measure the distance between two probability distributions.

Metric	Good GAN	Bad GAN
Inception Score ↑	1.153	1.345
FID ↓	5.326	38.828
Wasserstein ↓	0.164	0.712



Network Visualisation

playground.tensorflow.org

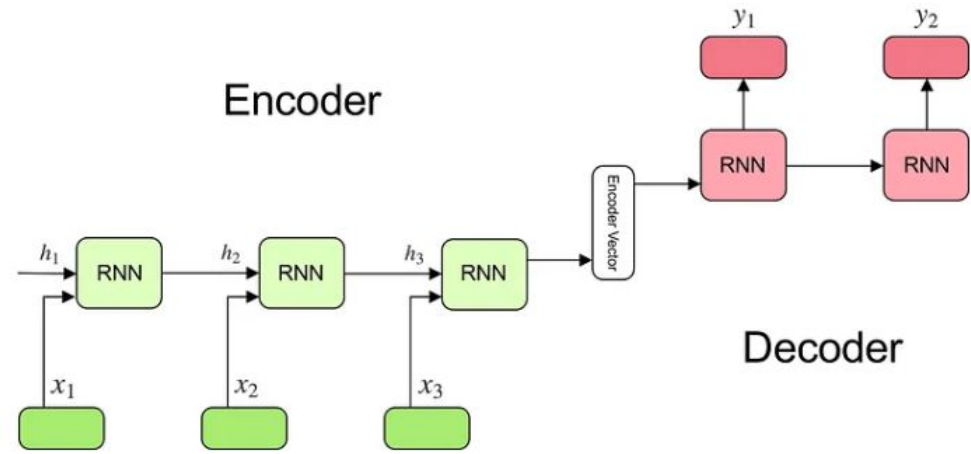
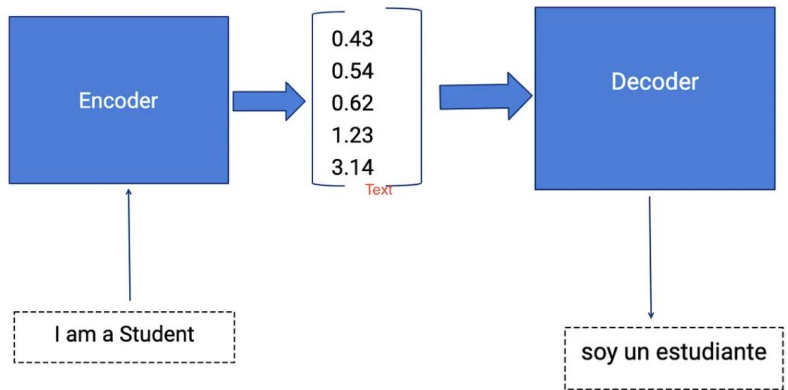
<https://poloclub.github.io/cnn-explainer/>

<https://poloclub.github.io/ganlab/>

<https://distill.pub/2021/gnn-intro/>

Encoder-decoder is a type of neural network architecture used for sequential data processing and generation consisting of three components:

- the encoder,
- the context vector, and
- the decoder



Encoder-decoder sequence to sequence model

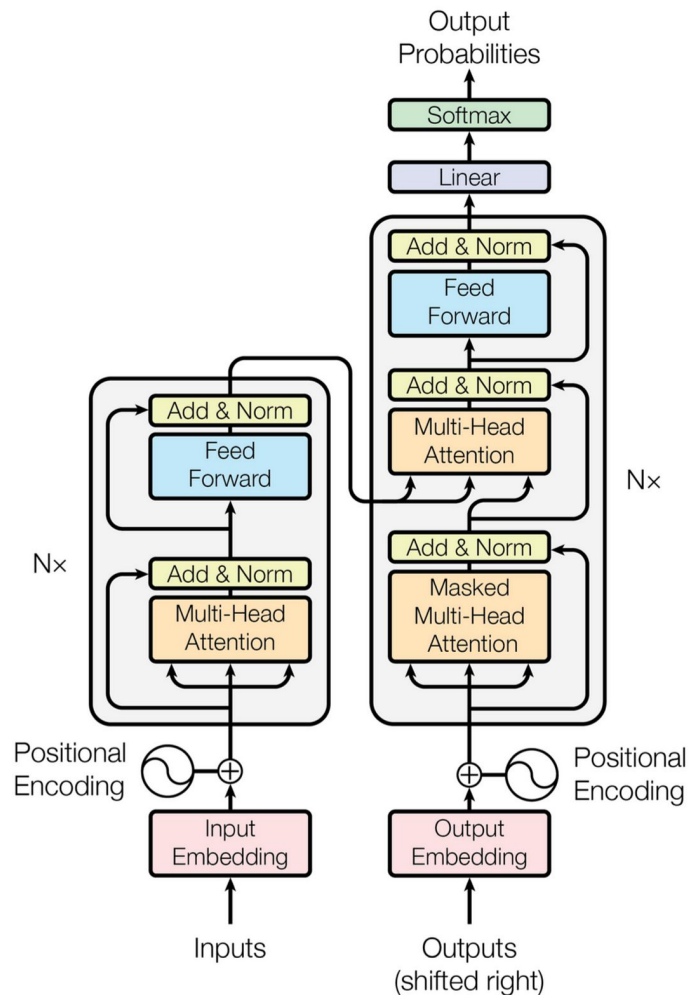
Transformer architecture

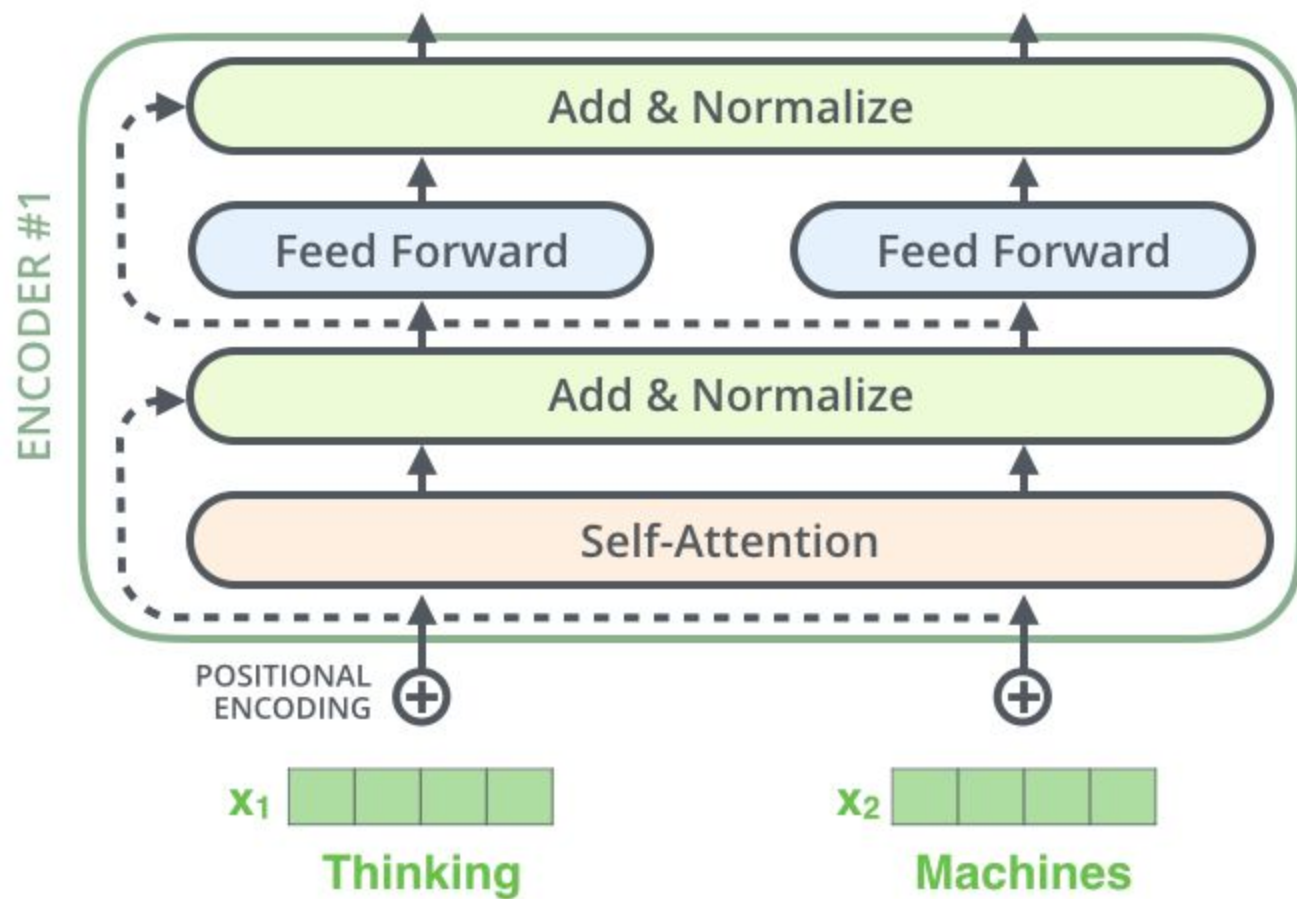
Encoder:

- Positional Encoding
- Attention
- Feedforward network

Decoder

- Positional encoding
- Self attention
- Cross attention
- Feedforward network

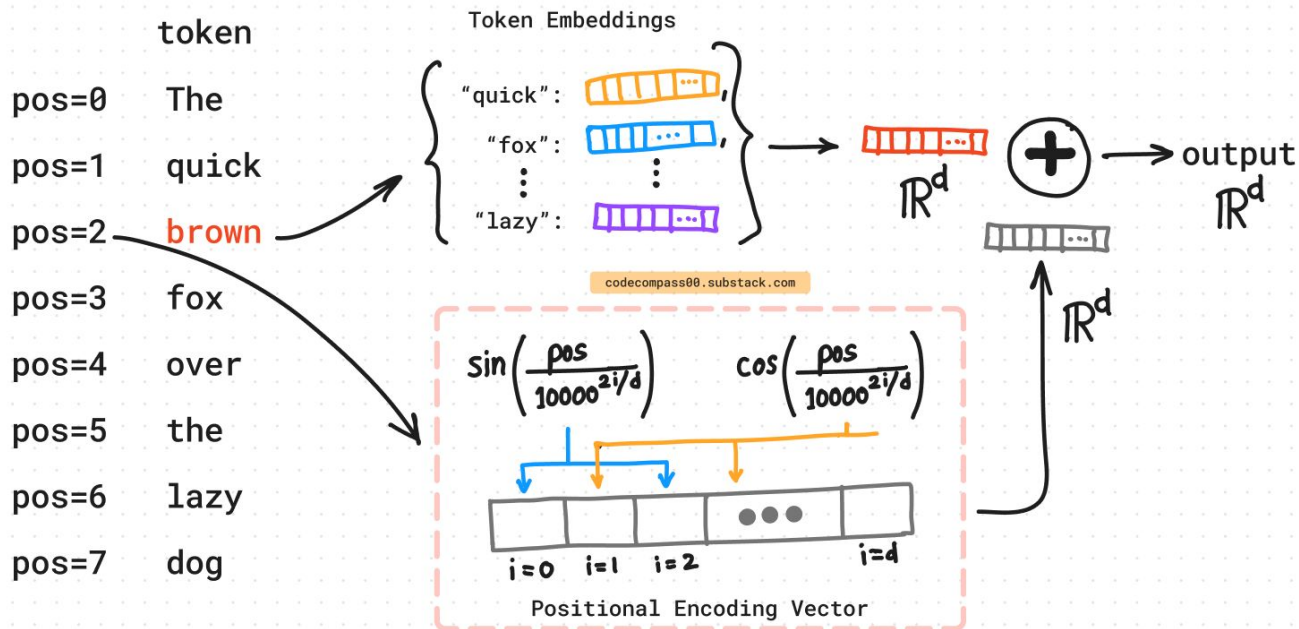




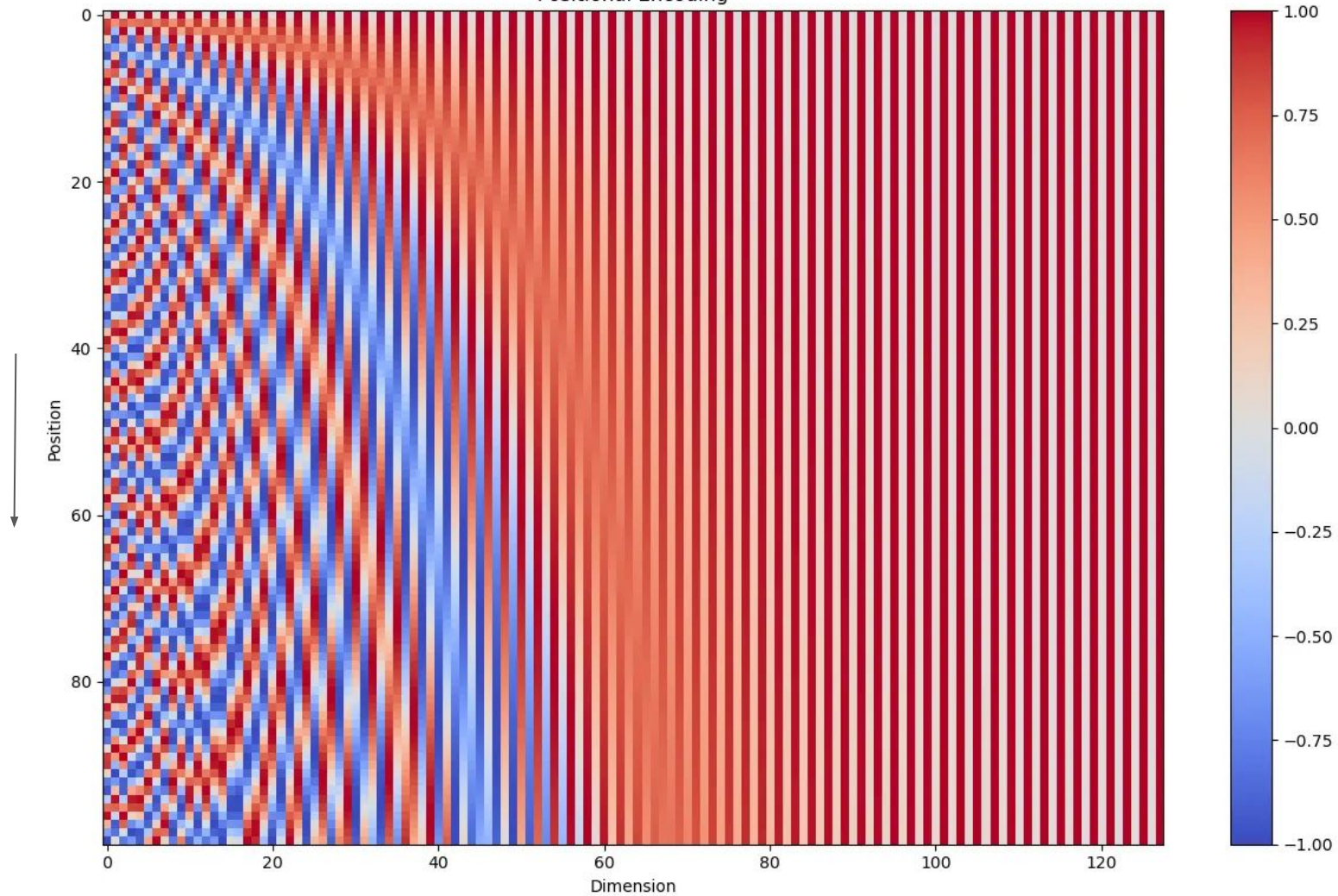
Transformer Encoder

Positional encoding

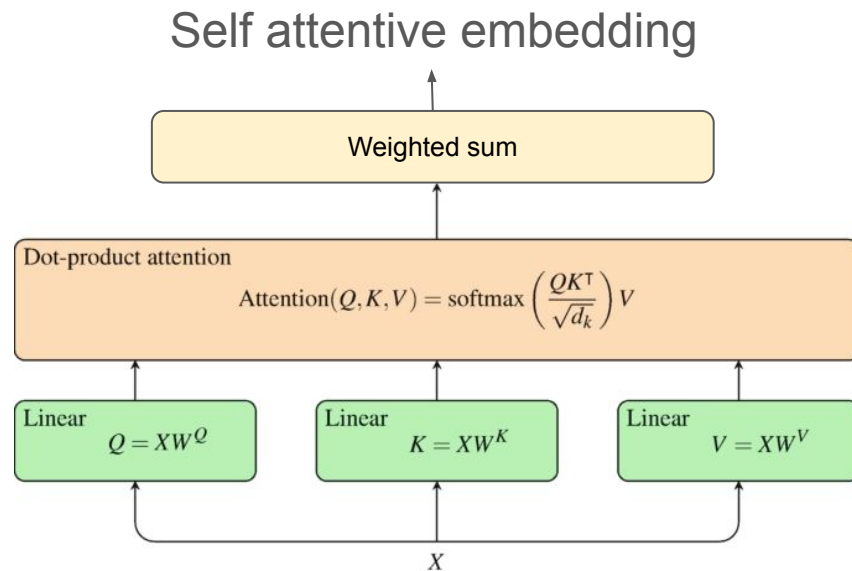
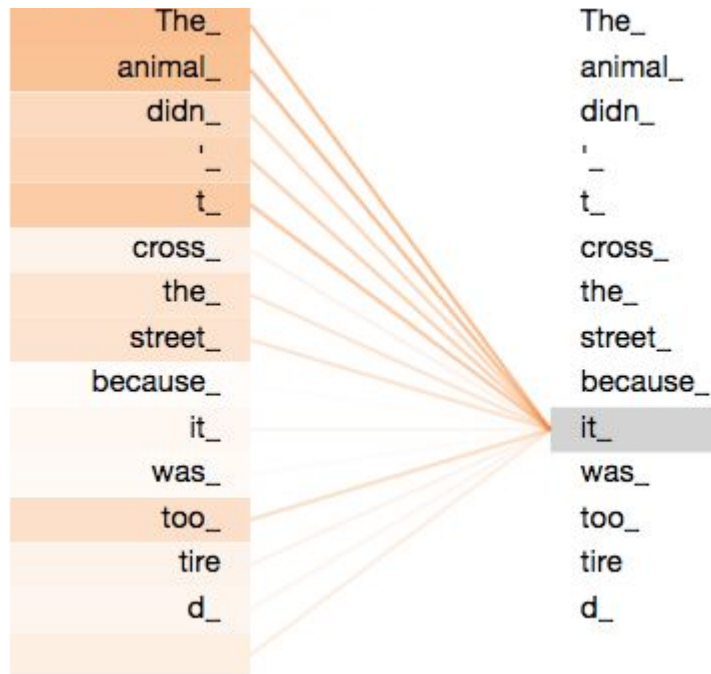
- Transformer treats all tokens equally and has no sense of sequence order. “man bites dog”
- Add a **positional encoding vector** to each token embedding before it enters the Transformer.
- Every token embedding now carries **both content and position**, allowing the model to understand order and relative distance.



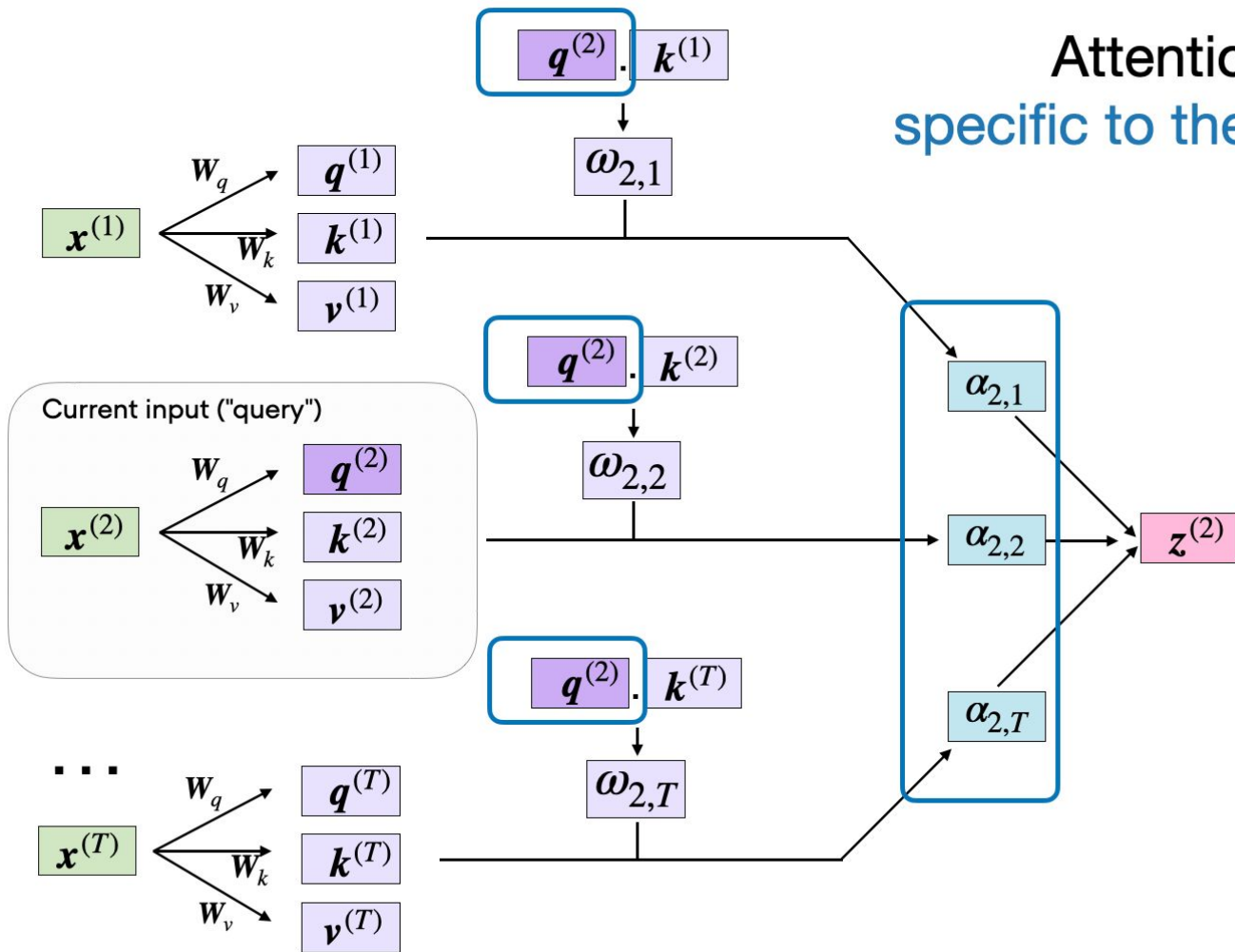
Positional Encoding



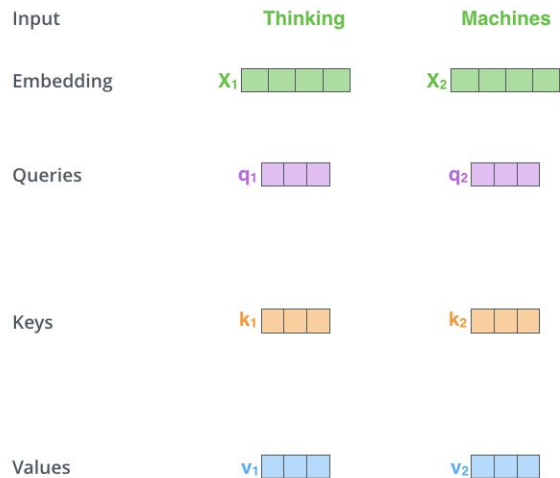
Self attention



Attention weights are
specific to the current input token



$$\text{where } z^{(2)} = \sum_{j=1}^T \alpha_{2,j} v^{(j)}$$


 W^Q

 W^K

 W^V

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

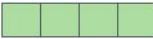
Softmax

X

Value

Sum $v_1 + v_2$

Thinking

x_1 

q_1 

k_1 

v_1 

$q_1 \cdot k_1 = 112$

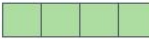
14

0.88

v_1 

z_1 

Machines

x_2 

q_2 

k_2 

v_2 

$q_2 \cdot k_2 = 96$

12

0.12

v_2 

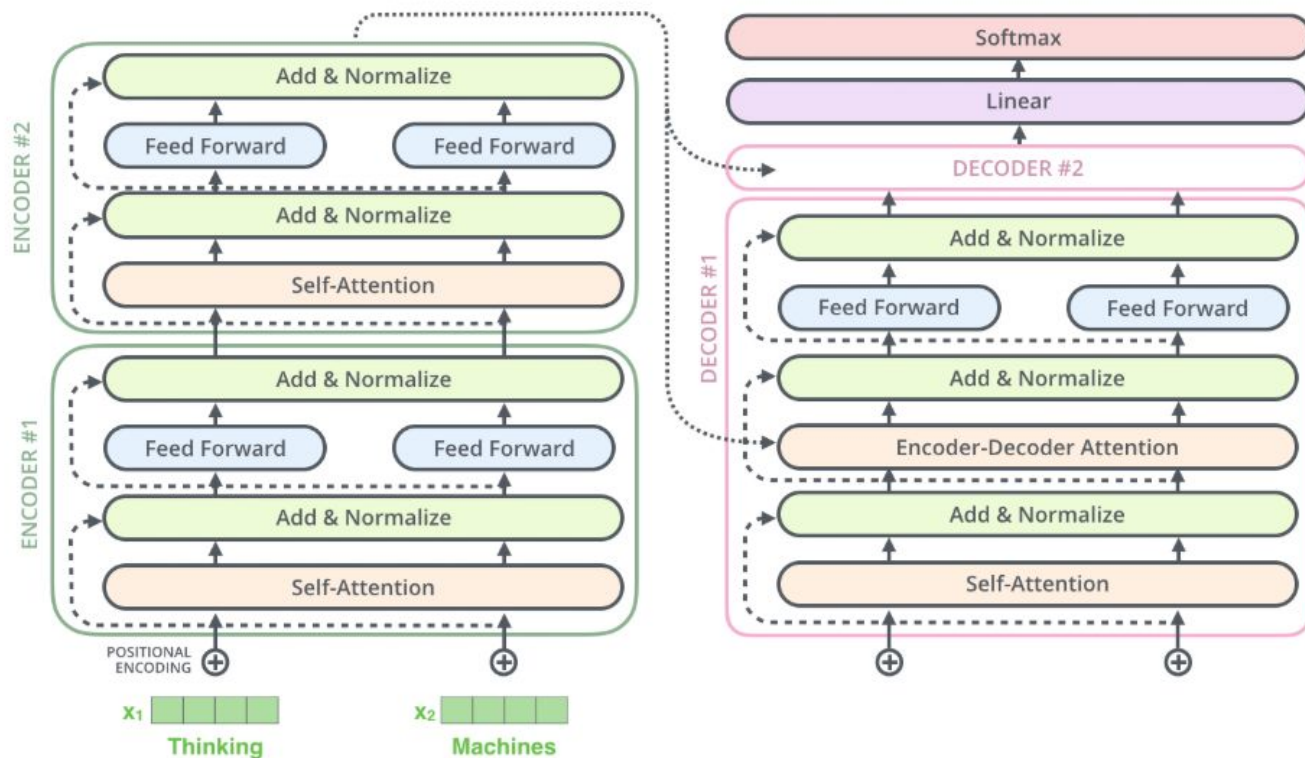
z_2 

Decoder

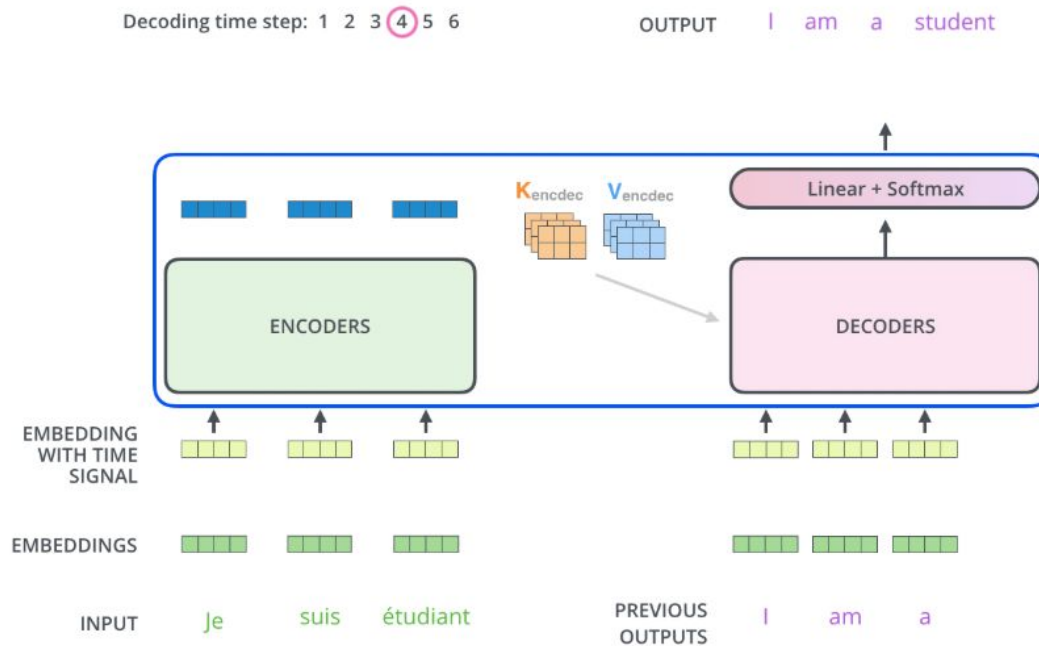
Cross attention

Query (Q) → comes from the **decoder's current hidden states** (the output of the decoder's self-attention and feed-forward layers).

Key (K) & Value (V) → come from the **encoder's final hidden states** (the encoder output sequence).



Decoder



<https://jalammar.github.io/illustrated-transformer/>